

Multi-Branch Smart Inventory & Order Management System

A concise technical overview of a modular-monolith full-stack system (Laravel 10 + Vue 3 SPA) built to support independent multi-branch operations, strict concurrency control, and enterprise-grade data integrity. Prepared for technical evaluation.



Problem Statement

Fragmented inventory per branch

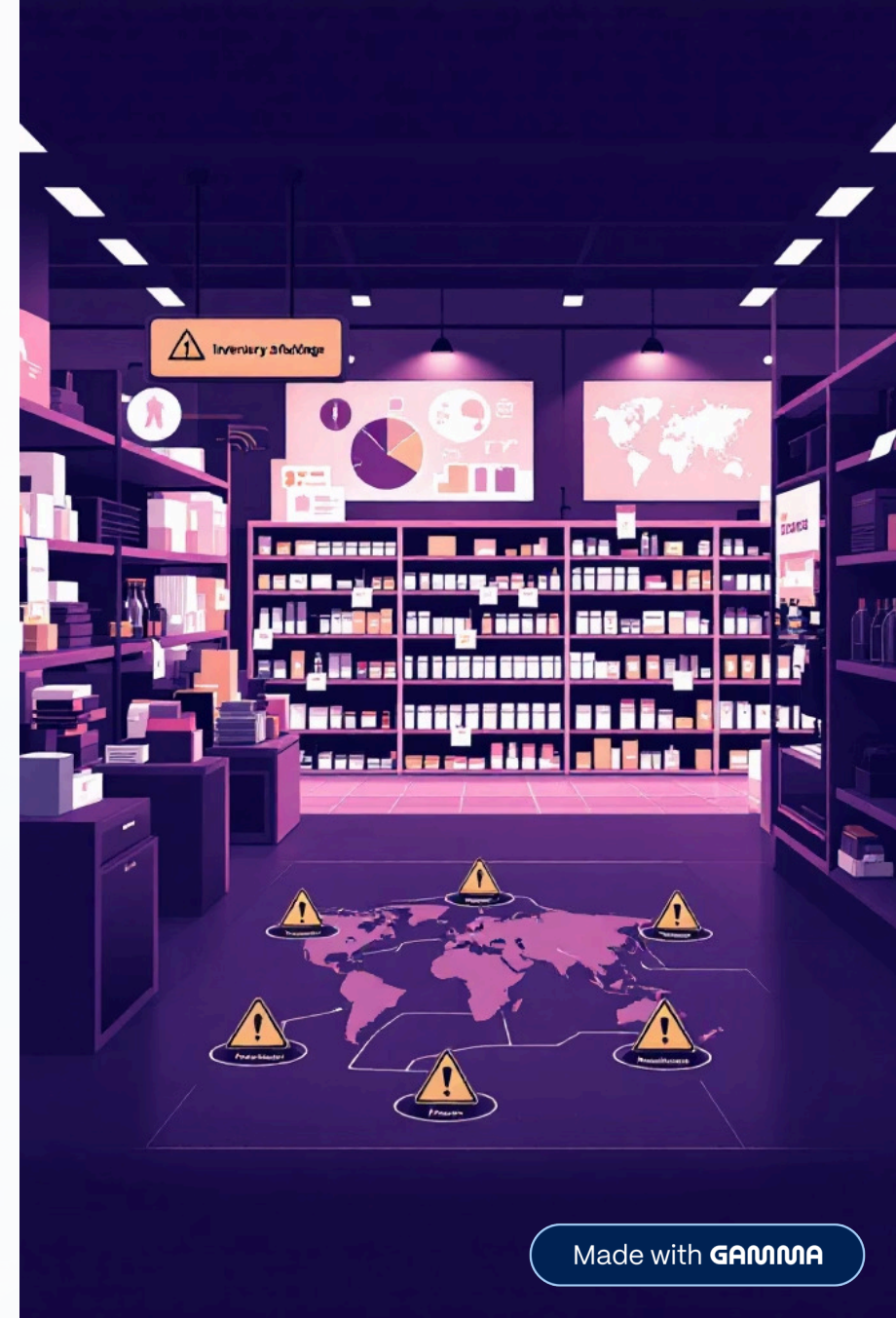
Branches must operate independently while allowing centralized oversight and reporting.

Race conditions & oversells

Concurrent sales can cause inconsistent stock unless transactions and row-level locking are enforced.

Need for predictable order lifecycle

Clear states (Draft → Confirmed → Completed → Cancelled) with deterministic side effects on inventory.

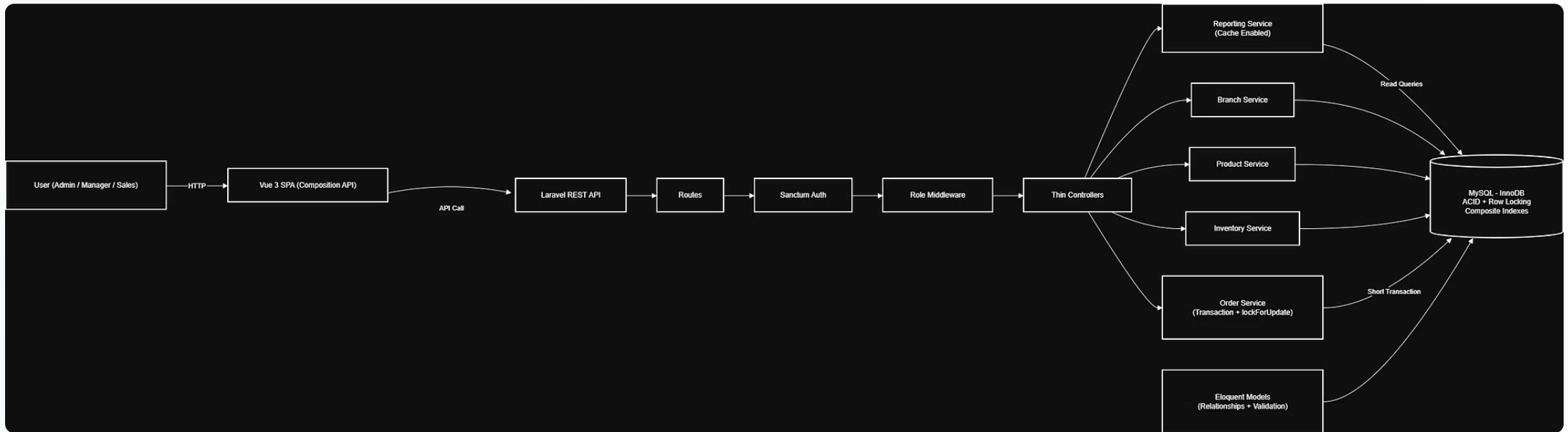


System Goals



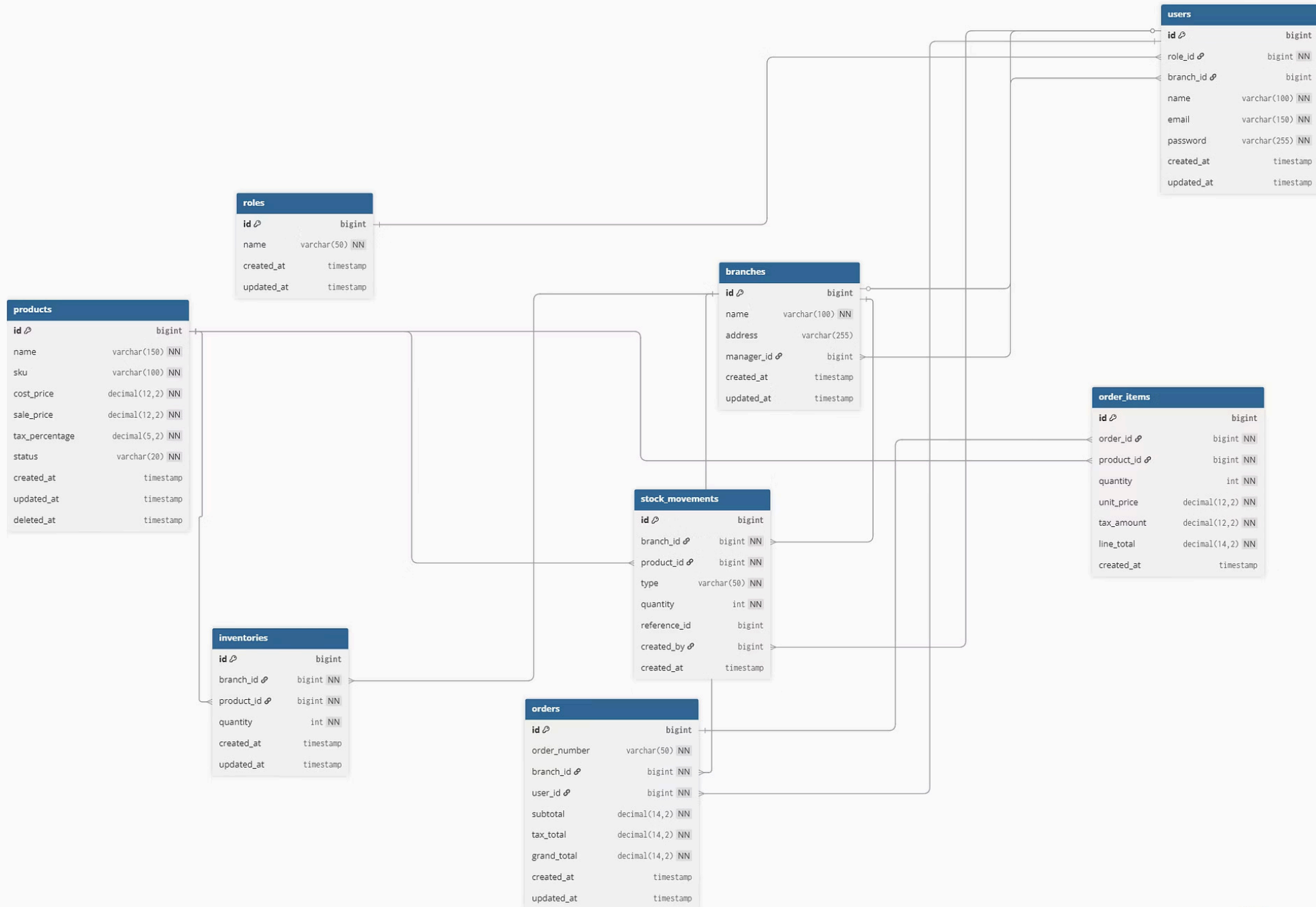
- Data integrity: ACID transactions for inventory & orders
- Concurrency safety: Prevent overselling using `lockForUpdate()`
- Branch isolation: Composite keys and scoped queries
- Maintainability: Thin controllers, fat service layer
- Scalability-ready: caching, read-replicas, horizontal-friendly

Architecture Overview

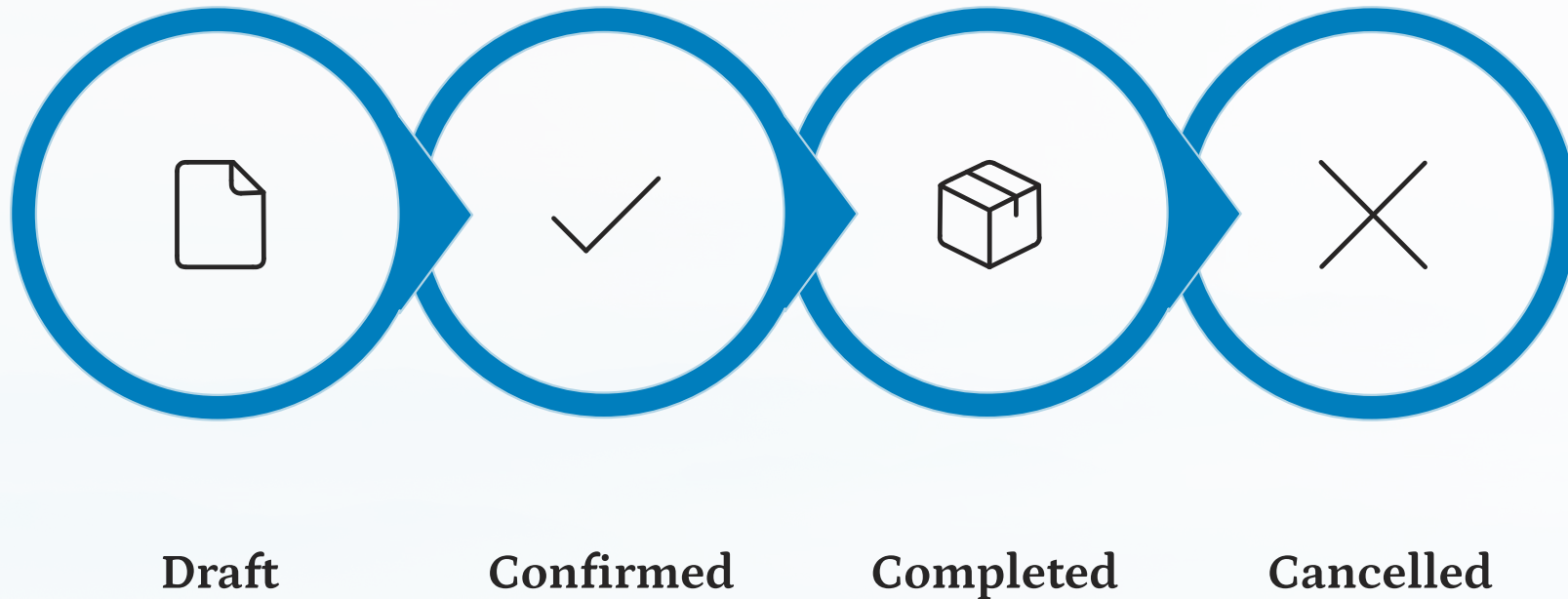


Layers enforce separation of concerns. Controllers handle request/response; Services encapsulate business rules; models map to DB with constraints. Sanctum provides token auth and middleware ensures role & branch isolation.

Database Schema — Key Tables

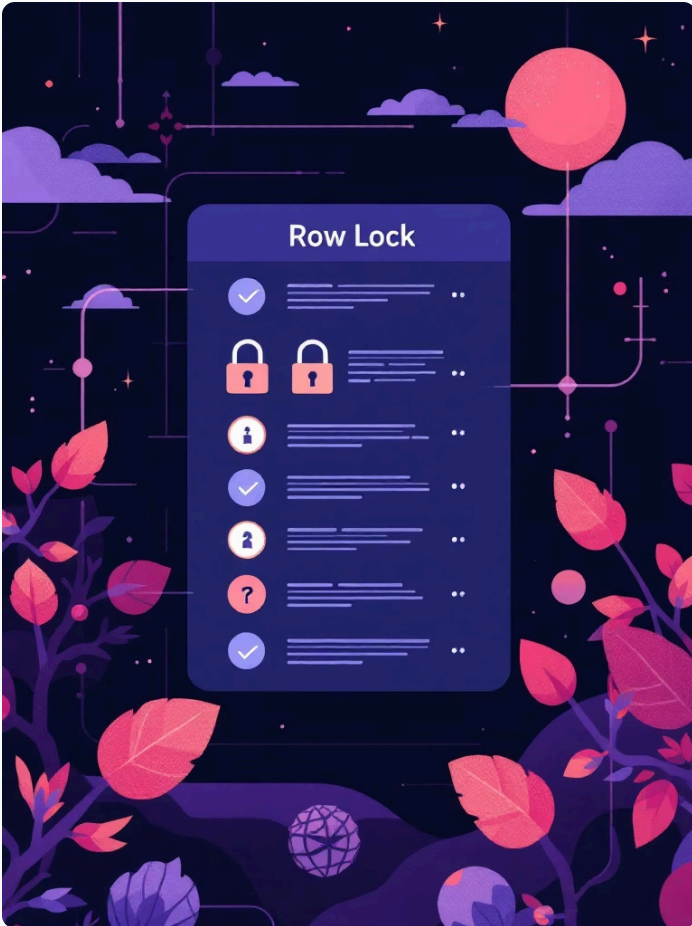


Order Lifecycle Flow



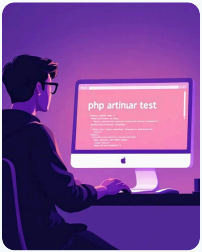
Transitions are atomic: each critical step runs inside DB transactions to maintain ACID properties and to ensure consistent stock accounting across branches.

Concurrency Handling — Implementation Details



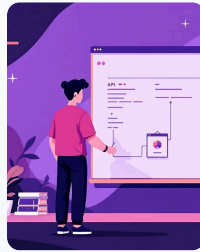
- Short transactions: limit lock duration to essential operations
- Row-level locking: Eloquent -> `DB::transaction + SELECT ... FOR UPDATE (lockForUpdate())`
- Reserve then deduct pattern:
 - a. Confirm: check `available = quantity_on_hand - quantity_reserved`
 - b. If sufficient: increment `quantity_reserved`
 - c. Complete: decrement `quantity_on_hand` and decrement `quantity_reserved`
- Indexing: `index(branch_id, product_id)` to make locking queries efficient

Service Layer & API Design



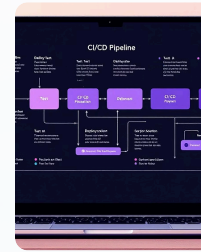
OrderService

Encapsulates order transitions, validation, stock reservation logic, and publishes domain events for reporting.



InventoryService

Atomic stock adjustments, stock_movements creation, returning consistent snapshots for dashboards.



ReportingService

Read-models and aggregated queries (prepared for migration to read replicas or materialized views).

Access Control & Security



- Authentication: Laravel Sanctum tokens for SPA; refresh-safe short-lived tokens recommended for production.
- Authorization: Role-based middleware (Super Admin, Branch Manager, Sales User) + branch scope checks on queries.
- Data isolation: All queries filtered by branch_id; composite indexes to enforce fast scoping.
- DB-level constraints: foreign keys + transactions to prevent orphaned records.

